

Gitlab CI/CD pipeline flow

Tools are used:

- ✓ Ubuntu/Amazon Linux
- ✓ Terraform
- ✓ Git
- ✓ GitLab
- ✓ EKS
- ✓ ARGOCD

Launch EC2(Jumpserver/Basiston) instance with pre-requisites:

Note : Install **aws cli** and **aws configure** to set **Accesskey** and **Secretkey** and add **Elastic IP** to the Jenkins server(**Optional but in Production/Dev is must**)

USED GITLAB TO LAUCH EKS CLUSTER

Terraform (EKS) code:

GitLab : <https://gitlab.com/sibanando/eks.git>

The screenshot displays the GitLab web interface for the 'eks' project. The sidebar on the left contains navigation links for Project, Pinned, Issues, Merge requests, Manage, Plan, Code, Build, Secure, Deploy, Operate, Monitor, Analyze, and Settings. The main content area shows a list of files in the 'eks' branch, including 'elastic_deployment_in_eks', 'gitlab-ci.yml', '0-provider.tf', '1-vpc.tf', '10-eks-access', '11-ecr', '2-igw.tf', '3-subnets.tf', '4-nat.tf', '5-routes.tf', '6-eks.tf', '7-nodes.tf', '8-iam-oidc.tf', '9-iam-test.tf', and 'README.md'. The 'Last commit' column indicates that most files were uploaded as new files, while 'README.md' was the initial commit. The 'Last update' column shows the time since the last update, ranging from '10 minutes ago' to '1 hour ago'. The 'Project information' sidebar on the right shows 21 commits, 1 branch, 0 tags, and 15 KIB project storage.

Provider.tf

=====

```
provider "aws" {  
  region = "us-east-1"  
}
```

```

terraform {
  required_providers {
    aws = {
      source = "hashicorp/aws"
      version = "~> 3.0"
    }
  }
}

Vpc.tf
=====

resource "aws_vpc" "Pharma_VPC" {
  cidr_block = "10.0.0.0/16"

  tags = {
    Name = "Pharma_VPC"
  }
}

Eks-access.tf
=====

resource "aws_eks_access_entry" "demo" {
  cluster_name    = aws_eks_cluster.demo.name
  principal_arn   = aws_iam_role.demo.arn
  kubernetes_groups = ["group-1", "group-2"]
  type            = "STANDARD"
}

Ecr.tf
=====

resource "aws_ecr_repository" "pharam_ecr_repo" {
  name          = "pharam_ecr_repo"
  image_tag_mutability = "MUTABLE"
  image_scanning_configuration {

```

```

    scan_on_push = true
  }
  tags = {
    Name = "pharam_ecr_repo"
  }
}

Dynamo-db.tf
=====

resource "aws_dynamodb_table" "terraform_lock" {
  name      = "Pharma_lock"
  billing_mode = "PAY_PER_REQUEST"
  hash_key  = "LockID"

  attribute {
    name = "LockID"
    type = "S"
  }
}

lgw.tf
=====

resource "aws_internet_gateway" "Pharma_igw" {
  vpc_id = aws_vpc.Pharma_VPC.id

  tags = {
    Name = "Pharma_igw"
  }
}

Subnets.tf
=====

resource "aws_subnet" "pharma_private-us-east-1a" {
  vpc_id      = aws_vpc.Pharma_VPC.id
  cidr_block  = "10.0.0.0/19"

```

```
availability_zone = "us-east-1a"
```

```
tags = {
```

```
    "Name" = "pharma_private-us-east-1a"
```

```
    "kubernetes.io/role/internal-elb" = "1"
```

```
    "kubernetes.io/cluster/demo" = "owned"
```

```
}
```

```
}
```

```
resource "aws_subnet" "pharma_private-us-east-1b" {
```

```
    vpc_id = aws_vpc.Pharma_VPC.id
```

```
    cidr_block = "10.0.32.0/19"
```

```
    availability_zone = "us-east-1b"
```

```
tags = {
```

```
    "Name" = "pharma_private-us-east-1b"
```

```
    "kubernetes.io/role/internal-elb" = "1"
```

```
    "kubernetes.io/cluster/demo" = "owned"
```

```
}
```

```
}
```

```
resource "aws_subnet" "pharma_public-us-east-1a" {
```

```
    vpc_id = aws_vpc.Pharma_VPC.id
```

```
    cidr_block = "10.0.64.0/19"
```

```
    availability_zone = "us-east-1a"
```

```
    map_public_ip_on_launch = true
```

```
tags = {
```

```
    "Name" = "pharma_public-us-east-1a"
```

```
    "kubernetes.io/role/elb" = "1"
```

```
    "kubernetes.io/cluster/demo" = "owned"
```

```
}
```

```
}
```

```
resource "aws_subnet" "pharma_public-us-east-1b" {
```

```
  vpc_id          = aws_vpc.Pharma_VPC.id
```

```
  cidr_block      = "10.0.96.0/19"
```

```
  availability_zone = "us-east-1b"
```

```
  map_public_ip_on_launch = true
```

```
  tags = {
```

```
    "Name"          = "pharma_public-us-east-1b"
```

```
    "kubernetes.io/role/elb" = "1"
```

```
    "kubernetes.io/cluster/demo" = "owned"
```

```
  }
```

```
}
```

```
Nat.tf
```

```
=====
```

```
resource "aws_eip" "pharma_nat" {
```

```
  vpc = true
```

```
  tags = {
```

```
    Name = "pharma_nat"
```

```
  }
```

```
}
```

```
resource "aws_nat_gateway" "pharma_nat" {
```

```
  allocation_id = aws_eip.pharma_nat.id
```

```
  subnet_id     = aws_subnet.pharma_public-us-east-1a.id
```

```
  tags = {
```

```
    Name = "pharma_nat"
```

```
  }
```

```

    depends_on = [aws_internet_gateway.Pharma_igw]
}

Routes.tf
=====

resource "aws_route_table" "pharma_private" {

    vpc_id = aws_vpc.Pharma_VPC.id

    route = [

        {

            cidr_block          = "0.0.0.0/0"

            nat_gateway_id      = aws_nat_gateway.pharma_nat.id

            carrier_gateway_id   = ""

            destination_prefix_list_id = ""

            egress_only_gateway_id = ""

            gateway_id          = ""

            instance_id          = ""

            ipv6_cidr_block      = ""

            local_gateway_id     = ""

            network_interface_id = ""

            transit_gateway_id   = ""

            vpc_endpoint_id      = ""

            vpc_peering_connection_id = ""

        },

    ]

    tags = {

        Name = "pharma_private"

    }

}

resource "aws_route_table" "pharma_public" {

    vpc_id = aws_vpc.Pharma_VPC.id

```

```

route = [
{
  cidr_block      = "0.0.0.0/0"
  gateway_id      = aws_internet_gateway.Pharma_igw.id
  nat_gateway_id  = ""
  carrier_gateway_id  = ""
  destination_prefix_list_id = ""
  egress_only_gateway_id  = ""
  instance_id      = ""
  ipv6_cidr_block  = ""
  local_gateway_id  = ""
  network_interface_id  = ""
  transit_gateway_id  = ""
  vpc_endpoint_id  = ""
  vpc_peering_connection_id = ""
},
]

```

```

tags = {
  Name = "pharma_public"
}
}

```

```

resource "aws_route_table_association" "pharma_private-us-east-1a" {
  subnet_id    = aws_subnet.pharma_private-us-east-1a.id
  route_table_id = aws_route_table.pharma_private.id
}

```

```

resource "aws_route_table_association" "pharma_private-us-east-1b" {
  subnet_id    = aws_subnet.pharma_private-us-east-1b.id
  route_table_id = aws_route_table.pharma_private.id
}

```



```
}
```

```
resource "aws_route_table_association" "pharma_public-us-east-1a" {  
  subnet_id    = aws_subnet.pharma_public-us-east-1a.id  
  route_table_id = aws_route_table.pharma_public.id  
}
```

```
resource "aws_route_table_association" "pharma_public-us-east-1b" {  
  subnet_id    = aws_subnet.pharma_public-us-east-1b.id  
  route_table_id = aws_route_table.pharma_public.id  
}
```

```
Eks.tf
```

```
=====
```

```
resource "aws_iam_role" "demo" {  
  name = "eks-cluster-demo-1"
```

```
  assume_role_policy = <<POLICY  
{  
  "Version": "2012-10-17",  
  "Statement": [  
    {  
      "Effect": "Allow",  
      "Principal": {  
        "Service": "eks.amazonaws.com"  
      },  
      "Action": "sts:AssumeRole"  
    }  
  ]  
}  
POLICY  
}
```

```
resource "aws_iam_role_policy_attachment" "demo-AmazonEKSClusterPolicy" {  
  policy_arn = "arn:aws:iam::aws:policy/AmazonEKSClusterPolicy"  
  role      = aws_iam_role.demo.name  
}
```

```
resource "aws_eks_cluster" "demo" {  
  name = "demo"  
  role_arn = aws_iam_role.demo.arn
```

```
  vpc_config {  
    subnet_ids = [  
      aws_subnet.pharma_private-us-east-1a.id,  
      aws_subnet.pharma_private-us-east-1b.id,  
      aws_subnet.pharma_public-us-east-1a.id,  
      aws_subnet.pharma_public-us-east-1b.id  
    ]  
  }  
}
```

```
  depends_on = [aws_iam_role_policy_attachment.demo-AmazonEKSClusterPolicy]  
}
```

Nodes.tf

=====

```
resource "aws_iam_role" "nodes" {  
  name = "eks-node-group-nodes-01"
```

```
  assume_role_policy = jsonencode({  
    Statement = [{  
      Action = "sts:AssumeRole"  
      Effect = "Allow"  
      Principal = {  
        Service = "ec2.amazonaws.com"  
      }  
    }  
  ]  
}
```

```

    }}
    Version = "2012-10-17"
  }}
}

resource "aws_iam_role_policy_attachment" "nodes-AmazonEKSWorkerNodePolicy" {
  policy_arn = "arn:aws:iam::aws:policy/AmazonEKSWorkerNodePolicy"
  role      = aws_iam_role.nodes.name
}

resource "aws_iam_role_policy_attachment" "nodes-AmazonEKS_CNI_Policy" {
  policy_arn = "arn:aws:iam::aws:policy/AmazonEKS_CNI_Policy"
  role      = aws_iam_role.nodes.name
}

resource "aws_iam_role_policy_attachment" "nodes-AmazonEC2ContainerRegistryReadOnly" {
  policy_arn = "arn:aws:iam::aws:policy/AmazonEC2ContainerRegistryReadOnly"
  role      = aws_iam_role.nodes.name
}

resource "aws_eks_node_group" "private-nodes" {
  cluster_name = aws_eks_cluster.demo.name
  node_group_name = "private-nodes"
  node_role_arn = aws_iam_role.nodes.arn

  subnet_ids = [
    aws_subnet.pharma_private-us-east-1a.id,
    aws_subnet.pharma_private-us-east-1b.id
  ]

  capacity_type = "ON_DEMAND"
  instance_types = ["t3.small"]

```

```
scaling_config {  
    desired_size = 1  
    max_size     = 3  
    min_size     = 1  
}
```

```
update_config {  
    max_unavailable = 1  
}
```

```
labels = {  
    role = "general"  
}
```

```
# taint {  
#   key   = "team"  
#   value = "devops"  
#   effect = "NO_SCHEDULE"  
# }
```

```
# launch_template {  
#   name   = aws_launch_template.eks-with-disks.name  
#   version = aws_launch_template.eks-with-disks.latest_version  
# }
```

```
depends_on = [  
    aws_iam_role_policy_attachment.nodes-AmazonEKSWorkerNodePolicy,  
    aws_iam_role_policy_attachment.nodes-AmazonEKS_CNI_Policy,  
    aws_iam_role_policy_attachment.nodes-AmazonEC2ContainerRegistryReadOnly,  
]  
}
```

```

# resource "aws_launch_template" "eks-with-disks" {
#   name = "eks-with-disks"

#   key_name = "local-provisioner"

#   block_device_mappings {
#     device_name = "/dev/xvdb"

#     ebs {
#       volume_size = 50
#       volume_type = "gp2"
#     }
#   }
# }

iam_oidc.tf
=====
data "tls_certificate" "eks" {
  url = aws_eks_cluster.demo.identity[0].oidc[0].issuer
}

resource "aws_iam_openid_connect_provider" "eks" {
  client_id_list = ["sts.amazonaws.com"]
  thumbprint_list = [data.tls_certificate.eks.certificates[0].sha1_fingerprint]
  url            = aws_eks_cluster.demo.identity[0].oidc[0].issuer
}

iam-test.tf
=====
data "aws_iam_policy_document" "test_oidc_assume_role_policy" {
  statement {
    actions = ["sts:AssumeRoleWithWebIdentity"]
    effect  = "Allow"
  }
}

```

```

condition {
  test    = "StringEquals"

  variable = "${replace(aws_iam_openid_connect_provider.eks.url, "https://", "")}:sub"
  values   = ["system:serviceaccount:default:aws-test"]
}

principals {
  identifiers = [aws_iam_openid_connect_provider.eks.arn]
  type       = "Federated"
}
}
}

resource "aws_iam_role" "test_oidc" {
  assume_role_policy = data.aws_iam_policy_document.test_oidc_assume_role_policy.json
  name               = "test-oidc"
}

resource "aws_iam_policy" "test-policy-1" {
  name = "test-policy-1"

  policy = jsonencode({
    Statement = [{
      Action = [
        "s3:ListAllMyBuckets",
        "s3:GetBucketLocation"
      ]
      Effect = "Allow"
      Resource = "arn:aws:s3:::*"
    }]
  })

  Version = "2012-10-17"

```

```

    })
}

resource "aws_iam_role_policy_attachment" "test_attach" {

    role      = aws_iam_role.test_oidc.name

    policy_arn = aws_iam_policy.test-policy-1.arn
}

```

```

output "test_policy_arn" {

    value = aws_iam_role.test_oidc.arn
}

```

Backend.tf

=====

```

terraform {

    backend "s3" {

        bucket = "testayush"

        key  = "pharma/terraform.tfstate"

        region = "us-east-1"

        encrypt      = true

        dynamodb_table = "terraform-lock"

    }

}

```

=====

GitLab pipeline code:

=====

.gitlab-ci.yml

=====

```

image:
  name: registry.gitlab.com/gitlab-org/gitlab-build-images:terraform
  entrypoint:
    - '/usr/bin/env'
    - 'PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin'

```

```

variables:
  TF_VAR_gitlab_token: ${TOKEN}
  AWS_ACCESS_KEY_ID: ${AWS_ACCESS_KEY_ID}
  AWS_SECRET_ACCESS_KEY : ${AWS_SECRET_ACCESS_KEY}
  AWS_DEFAULT_REGION: "us-east-1"

```

```
cache:
  paths:
    - .terraform

before_script:
  - terraform --version
  - terraform init

stages:
  - validate
  - plan
  - apply
  - destroy

validate:
  stage: validate
  script:
    - terraform validate

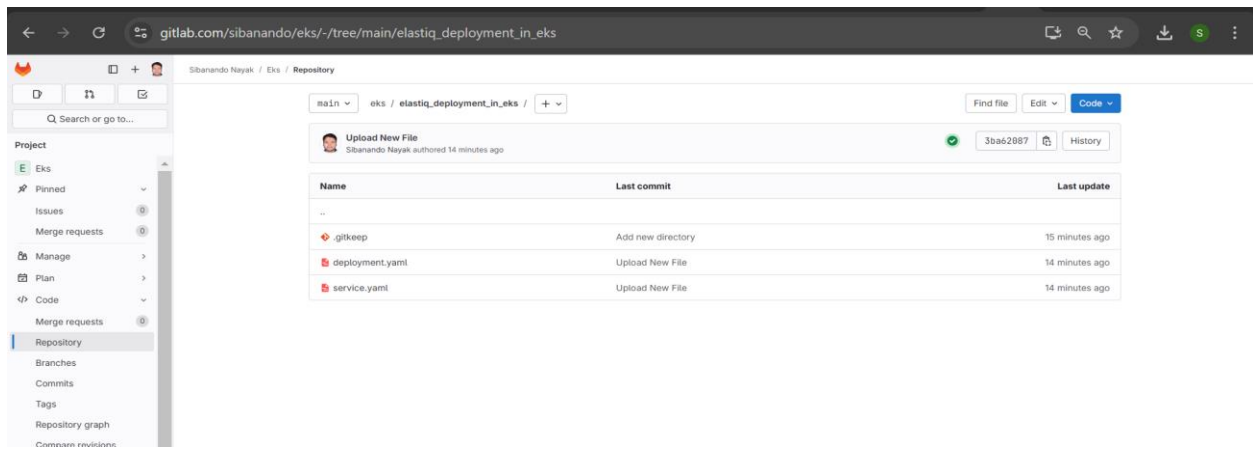
plan:
  stage: plan
  script:
    - terraform plan -out="planfile"
  dependencies:
    - validate
  artifacts:
    paths:
      - planfile

apply:
  stage: apply
  script:
    - terraform apply -input=false "planfile"
  dependencies:
    - plan
  when: manual

destroy:
  stage: destroy
  script:
    - terraform destroy --auto-approve
  when: manual
```


Deployment in EKS

https://gitlab.com/sibanando/eks/-/tree/main/elastiq_deployment_in_eks



=====

Deployment in EKS using ARGOCD

=====

```
^Croot@ip-172-31-17-160:~# kubectl get pods -n argocd
NAME                                READY   STATUS    RESTARTS   AGE
argocd-application-controller-0     1/1     Running   0           54s
argocd-applicationset-controller-64f6bd6456-wkf87  1/1     Running   0           55s
argocd-dex-server-5fdcd9df8b-9p868  1/1     Running   0           55s
argocd-notifications-controller-778495d96f-z5xsc  1/1     Running   0           55s
argocd-redis-69fd8bd669-9xzmw       1/1     Running   0           55s
argocd-repo-server-75567c944-tx2pr   1/1     Running   0           55s
argocd-server-5c768cdd96-fnd9s      1/1     Running   0           55s
root@ip-172-31-17-160:~# kubectl patch svc argocd-server -n argocd -p '{"spec": {"type": "LoadBalancer"}}'
service/argocd-server patched
```

```
root@ip-172-31-17-160:~# echo eil6QThGdFk2U09Vc3hVZW== | base64 --decode
z-zA8FtY6sOUxkUgroot@ip-172-31-17-160:~# kubectl get svc -n argocd
NAME                                TYPE                CLUSTER-IP      EXTERNAL-IP      PORT(S)
argocd-applicationset-controller    ClusterIP           172.20.189.250   <none>            7000/TCP,8080/TCP
argocd-dex-server                   ClusterIP           172.20.71.177    <none>            5556/TCP,5557/TCP,5558/TCP
argocd-metrics                      ClusterIP           172.20.236.112   <none>            8082/TCP
argocd-notifications-controller-metrics ClusterIP           172.20.157.229   <none>            9001/TCP
argocd-redis                        ClusterIP           172.20.213.156   <none>            6379/TCP
argocd-repo-server                  ClusterIP           172.20.23.37     <none>            8081/TCP,8084/TCP
argocd-server                       LoadBalancer        172.20.12.59     a556c269c7db64473a41b20d5b42122a-1421385761.us-east-1.elb.amazonaws.com 80:30219/TCP,443:32594/TCP
argocd-server-metrics               ClusterIP           172.20.88.96     <none>            8083/TCP
root@ip-172-31-17-160:~#
```

The screenshot shows the Argo CD web interface. On the left is a sidebar with navigation links: Applications, Settings, User Info, and Documentation. Below these are filters for NAME, KINDS, and SYNC STATUS. The main content area is titled 'SUMMARY' and shows details for a service named 'helloworld' in the 'default' namespace. The service was created on 01/01/2025 at 18:08:45. Its type is 'LoadBalancer' and it has a status of 'Synced' and 'Healthy'. The hostnames are listed as 'ae98ede4da5b04ba3afb0c7da184b9-378378860.us-east-1.elb.amazonaws.com'. There are no links listed at the bottom.

KIND	Service
NAME	helloworld
NAMESPACE	default
CREATED AT	01/01/2025 18:08:45 (an hour ago)
TYPE	LoadBalancer
HOSTNAMES	ae98ede4da5b04ba3afb0c7da184b9-378378860.us-east-1.elb.amazonaws.com
STATUS	Synced
HEALTH	Healthy
LINKS	

URL:

